CHAPTER 3
Simulation Using ROS

3.1. Overview:

Simulation is the process of using a digital or mathematical model to represent a real-world system or phenomenon in order to study its behavior under different conditions. Simulations are widely used in various fields such as engineering, science, economics, and medical training. It contains a lot of plugins that is used for making our world and the characteristics of the frame of robot that need to be design and sensors and actuators related to object and communication between robot component and how to interact with the environment.

3.2. The main components of simulation

3.2.1 Gazebo

Gazebo is a powerful open-source robotics simulation software that provides a realistic and dynamic environment for testing and developing robotic systems. It offers a physics engine, sensor simulations, and visualization capabilities, allowing users to simulate and analyze the behavior of robots in various scenarios. Gazebo is widely used in research, education, and industry to design, prototype, and evaluate robotic systems before deploying them in the real world. A Physical Gazebo In the world of parks, gardens, and backyards, a gazebo is a freestanding structure with a roof and open sides. It's a decorative element that provides shade, shelter from the elements, and a pleasant spot to relax or entertain guests. Gazebo Simulator In the realm of robotics research and development, Gazebo refers to a powerful 3D robot simulator. Unlike a game engine, Gazebo prioritizes high-fidelity physics simulation to create realistic environments for testing robots. It offers features like Multiple physics engines Simulate robot behavior under different physical conditions. Variety of sensors Simulate real-world sensors like cameras, LiDAR, and accelerometers. User and programmatic interfaces: Interact with the simulation visually or through code.

Install gazebo for ubuntu 20.04

i) **Setup system to accept software from Index of / (osrfoundation.org):** sudo sh -c 'echo "deb http://packages.osrfoundation.org/gazebo/ubuntu-stable `lsb_release -cs` main" > /etc/apt/sources.list.d/gazebo-stable.list'

ii) **Setup keys:** wget http://packages.osrfoundation.org/gazebo.key -O - | sudo apt-key add –

iii) **Install gazebo:** sudo apt-get install gazebo11 libgazebo11-dev

iv) **Gazebo plugin for apm2.8:** cd ~ git clone https://github.com/khancyr/ardupilot_gazebo.git

cd ardupilot_gazebo

build our plugin:

mkdir build

cd build

cmake.

make -j4

sudo make install

3.2.2Ros

The robotics operating system that is used to connect sensors over messages and topics to communicate with the environment for our drone First, we need to install ros (noetic) that require linux (ubuntu 20.04) and install using series of command and frameworks**:**

i) 1.Setup your sources. list with command:

- **sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" > /etc/apt/sources.list.d/ros-latest.list'**

ii) install ROS (noetic):

```
sudo apt install ros-noetic-desktop-full
```

iii) we need to setup our environment:

```
source /opt/ros/noetic/setup.bash
```

iv) reload bashrc file in ubuntu

```
echo "source /opt/ros/noetic/setup.bash" >> ~/. bashrc
source ~/. bashrc
```

v) Dependencies for building packages

```
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-generator python 3-wstool build-essential
```

Second, we need to create our work space in which we will make our packages and building for our simulation. Packages that have been used are as follows:

1.IQ-SIM
2.IQ-GNC
3.Darknet-Ros

We need to setup our workspace by some few of steps:

Sudo apt-get install python-wstool python-rosinstall-generator python-catkin-tools

And then we will create a folder called drone work space and inside it we created a folder called source and the packages that were identified above will be uploaded to the source

Initialize our work space:

mkdir -p ~/drone_ws/src

cd ~/drone_ws

catkin init

Then we need to install some additional packages that will help us in identifying flight controller and the communication between the flight controller and raspberry pi controller for making a high processing technique like object detection.

The first additional package is mavros that is a package provides communication driver for various autopilots with MAVLink communication protocol. Additional it provides UDP MAVLink bridge for ground control stations (e.g., QGroundControl).

Main node can be extended by plugins (see pluginlib). See also mavros_extras package and we install it and building it in our drone work space using:

cd ~/drone_ws

wstool init ~/drone_ws/src

rosinstall_generator --upstream mavros | tee /tmp/mavros.rosinstall

rosinstall_generator mavlink | tee -a /tmp/mavros.rosinstall

wstool merge -t src /tmp/mavros.rosinstall

wstool update -t src

rosdep install --from-paths src --ignore-src --rosdistro `echo $ROS_DISTRO` -y

catkin build

echo "source ~/catkin_ws/devel/setup.bash" >> ~/. bashrc

We can reed areal telemetry from mavros topics and these is done by running roscore and see ros topic list that will be shown as:
/Clock
/gazebo/link_states
/gazebo/model_states
/gazebo/parameter_descriptions
/gazebo/parameter_updates
/gazebo/set_link_state
/gazebo/set_model_state
/rosout

/rosout_agg
/webcam/camera_info
/webcam/image_raw
/webcam/image_raw/compressed
/webcam/image_raw/compressed/parameter_descriptions
/webcam/image_raw/compressed/parameter_updates
/webcam/image_raw/compressedDepth
/webcam/image_raw/compressedDepth/parameter_descriptions
/webcam/image_raw/compressedDepth/parameter_updates
/webcam/image_raw/theora
/webcam/image_raw/theora/parameter_descriptions
/webcam/image_raw/theora/parameter_updates
/webcam/parameter_descriptions
/webcam/parameter_updates

Using MAVROS to get telemetry data from the FCU

now the topic /gazebo/model_states is the true model position in the simulator. This isn't something we can use in real life. In real life, we have to use the estimate of the drone's position which is formulated from a combination of its sensors. This position is transmitted using a communication protocol called mavlink.

These messages are stripped down and are optimized for radio transmission MAVROS is a middle man which translates the MAVlink messages into ROS messages, which are easy to use and common between different robot systems.

To start mavros run following commands

roslaunch iq_sim apm. launch

and then run rostopic again and will see a different masseages shown

/clock
/diagnostics
/gazebo/link_states
/gazebo/model_states
/gazebo/parameter_descriptions
/gazebo/parameter_updates
/gazebo/set_link_state
/gazebo/set_model_state
/mavlink/from
/mavlink/to
/mavros/adsb/send
/mavros/adsb/vehicle
/mavros/battery
/mavros/cam_imu_sync/cam_imu_stamp
/mavros/companion_process/status
/mavros/distance_sensor/rangefinder_pub
/mavros/distance_sensor/rangefinder_sub
/mavros/extended_state
/mavros/fake_gps/mocap/tf
/mavros/global_position/compass_hdg
/mavros/global_position/global
/mavros/global_position/gp_lp_offset
/mavros/global_position/gp_origin

/mavros/global_position/home
/mavros/global_position/local
/mavros/global_position/raw/fix
/mavros/global_position/raw/gps_vel
/mavros/global_position/rel_alt
/mavros/global_position/set_gp_origin
/mavros/gps_rtk/send_rtcm
/mavros/home_position/home
/mavros/home_position/set
/mavros/imu/data
/mavros/imu/data_raw
/mavros/imu/diff_pressure
/mavros/imu/mag
/mavros/imu/static_pressure
/mavros/imu/temperature_baro
/mavros/imu/temperature_imu
/mavros/landing_target/lt_marker
/mavros/landing_target/pose
/mavros/landing_target/pose_in
/mavros/local_position/accel
/mavros/local_position/odom
/mavros/local_position/pose
/mavros/local_position/pose_cov
/mavros/local_position/velocity_body
/mavros/local_position/velocity_body_cov
/mavros/local_position/velocity_local
/mavros/log_transfer/raw/log_data
/mavros/log_transfer/raw/log_entry
/mavros/manual_control/control
/mavros/manual_control/send
/mavros/mission/reached
/mavros/mission/waypoints
/mavros/mocap/pose
/mavros/obstacle/send
/mavros/odometry/in
/mavros/odometry/out
/mavros/param/param_value
/mavros/radio_status
/mavros/rangefinder/rangefinder
/mavros/rc/in
/mavros/rc/out
/mavros/rc/override
/mavros/setpoint_accel/accel
/mavros/setpoint_attitude/cmd_vel
/mavros/setpoint_attitude/thrust
/mavros/setpoint_position/global
/mavros/setpoint_position/local
/mavros/setpoint_raw/attitude
/mavros/setpoint_raw/global
/mavros/setpoint_raw/local

/mavros/setpoint_raw/target_attitude
/mavros/setpoint_raw/target_global
/mavros/setpoint_raw/target_local
/mavros/setpoint_velocity/cmd_vel
/mavros/setpoint_velocity/cmd_vel_unstamped
/mavros/state
/mavros/statustext/recv
/mavros/statustext/send
/mavros/time_reference
/mavros/timesync_status
/mavros/trajectory/desired
/mavros/trajectory/generated
/mavros/trajectory/path
/mavros/vfr_hud
/mavros/vision_pose/pose
/mavros/vision_pose/pose_cov
/mavros/wind_estimation
/rosout
/rosout_agg
/tf
/tf_static
/webcam/camera_info
/webcam/image_raw
/webcam/image_raw/compressed
/webcam/image_raw/compressed/parameter_descriptions
/webcam/image_raw/compressed/parameter_updates
/webcam/image_raw/compressedDepth
/webcam/image_raw/compressedDepth/parameter_descriptions
/webcam/image_raw/compressedDepth/parameter_updates
/webcam/image_raw/theora
/webcam/image_raw/theora/parameter_descriptions
/webcam/image_raw/theora/parameter_updates
/webcam/parameter_descriptions
/webcam/parameter_updates

Now we can see the drone's position in its local frame by running rostopic echo /mavros/global_position/local

In the following tutorials we will be accessing the data on these topics in our C++ programs. To see the type of message being published run

rostopic list -v /mavros/global_position/local

We see that the topic is publisng the message in the form of mav_msgs/Odometry

to see the structure of the message you can run the following command. rosmsg show nav_msgs/Odometry

This will be usefull when writing publishers and subscribers in the future

second additional package is mavlink it we need and to install to install mavproxy configuration protocal and install it using :

sudo pip install future pymavlink MAVProxy

And we need to edit our bashrc file to ensure that mavproxy add to the system**:**

gedit ~/. bashrc

export PATH=$PATH: $HOME/ardupilot/Tools/autotest

export PATH=/usr/lib/ccache: $PATH

. ~/. bashrc

Now, we can install mavlink for controlling our drone using:

```
git clone https://github.com/mavlink/mavlink.git –recursive

python3 -m pip install -r pymavlink/requirements.txt
```

We install everything to create our package for our drone and the world in gazebo let's do it the first package we will create is iq_sim

1.IQ-Sim

We needed to build our drone and the other world, and then we went in to make our model, which is a drone quadcopter, is a type of helicopter or multicopter that has four rotors. the frame for the drone, which we built by model.sdf file:

```xml
<?xml version='1.0'?>
<sdf version="1.6" >
  <model name="iris_demo">
    <include>
      <uri>model://iris_base</uri>
    </include>


    <!-- plugins -->
    <plugin name="rotor_0_blade_1" filename="libLiftDragPlugin.so">
      <a0>0.3</a0>
      <alpha_stall>1.4</alpha_stall>
      <cla>4.2500</cla>
      <cda>0.10</cda>
      <cma>0.00</cma>
      <cla_stall>-0.025</cla_stall>
      <cda_stall>0.0</cda_stall>
      <cma_stall>0.0</cma_stall>
      <area>0.002</area>
      <air_density>1.2041</air_density>
      <cp>0.084 0 0</cp>
      <forward>0 1 0</forward>
      <upward>0 0 1</upward>
      <link_name>iris::rotor_0</link_name>
    </plugin>
    <plugin name="rotor_0_blade_2" filename="libLiftDragPlugin.so">
      <a0>0.3</a0>
      <alpha_stall>1.4</alpha_stall>
      <cla>4.2500</cla>
      <cda>0.10</cda>
      <cma>0.00</cma>
      <cla_stall>-0.025</cla_stall>
      <cda_stall>0.0</cda_stall>
      <cma_stall>0.0</cma_stall>
      <area>0.002</area>
      <air_density>1.2041</air_density>
      <cp>-0.084 0 0</cp>
      <forward>0 -1 0</forward>
      <upward>0 0 1</upward>
      <link_name>iris::rotor_0</link_name>
    </plugin>

    <plugin name="rotor_1_blade_1" filename="libLiftDragPlugin.so">
      <a0>0.3</a0>
      <alpha_stall>1.4</alpha_stall>
      <cla>4.2500</cla>
      <cda>0.10</cda>
      <cma>0.00</cma>
      <cla_stall>-0.025</cla_stall>
      <cda_stall>0.0</cda_stall>
      <cma_stall>0.0</cma_stall>
      <area>0.002</area>
      <air_density>1.2041</air_density>
      <cp>0.084 0 0</cp>
```

```
        <i_max>0</i_max>
        <i_min>0</i_min>
        <cmd_max>2.5</cmd_max>
        <cmd_min>-2.5</cmd_min>
        <jointName>iris::rotor_0_joint</jointName>
        <multiplier>838</multiplier>
        <controlVelocitySlowdownSim>1</controlVelocitySlowdownSim>
      </control>
      <control channel="1">
        <type>VELOCITY</type>
        <offset>0</offset>
        <p_gain>0.20</p_gain>
        <i_gain>0</i_gain>
        <d_gain>0</d_gain>
        <i_max>0</i_max>
        <i_min>0</i_min>
        <cmd_max>2.5</cmd_max>
        <cmd_min>-2.5</cmd_min>
        <jointName>iris::rotor_1_joint</jointName>
        <multiplier>838</multiplier>
        <controlVelocitySlowdownSim>1</controlVelocitySlowdownSim>
      </control>
      <control channel="2">
        <type>VELOCITY</type>
        <offset>0</offset>
        <p_gain>0.20</p_gain>
        <i_gain>0</i_gain>
        <d_gain>0</d_gain>
        <i_max>0</i_max>
        <i_min>0</i_min>
        <cmd_max>2.5</cmd_max>
        <cmd_min>-2.5</cmd_min>
        <jointName>iris::rotor_2_joint</jointName>
        <multiplier>-838</multiplier>
        <controlVelocitySlowdownSim>1</controlVelocitySlowdownSim>
      </control>
      <control channel="3">
        <type>VELOCITY</type>
        <offset>0</offset>
        <p_gain>0.20</p_gain>
        <i_gain>0</i_gain>
        <d_gain>0</d_gain>
        <i_max>0</i_max>
        <i_min>0</i_min>
        <cmd_max>2.5</cmd_max>
        <cmd_min>-2.5</cmd_min>
        <jointName>iris::rotor_3_joint</jointName>
        <multiplier>-838</multiplier>
        <controlVelocitySlowdownSim>1</controlVelocitySlowdownSim>
      </control>
    </plugin>

  </model>
</sdf>
```

We needed to make our own world and we would create it in drone. world file that contains are as follows

1.gazebo world

2.Camera sensor

3.Lidear sensor

We need to make our world and insert our drone on it and that make by:

```
:?xml version="1.0" ?>
:sdf version="1.6">
  <world name="default">
    <physics type="ode">
      <ode>
        <solver>
          <type>quick</type>
          <iters>100</iters>
          <sor>1.0</sor>
        </solver>
        <constraints>
          <cfm>0.0</cfm>
          <erp>0.9</erp>
          <contact_max_correcting_vel>0.1</contact_max_correcting_vel>
          <contact_surface_layer>0.0</contact_surface_layer>
        </constraints>
      </ode>
      <real_time_update_rate>-1</real_time_update_rate>
      <!-- <max_step_size>0.0020</max_step_size> -->
    </physics>

    <include>
      <uri>model://sun</uri>
    </include>

    <include>
      <uri>model://ground_plane</uri>
    </include>

    <model name="iris">
      <include>
        <uri>model://iris_with_standoffs_demo</uri>
      </include>
```

First sensor is camera we need to identify

Camera's (link, joint)

```
<link name='camera'>
  <pose>0 -0.01 0.070 1.57 0 1.57</pose>
  <inertial>
    <pose>0 0 0 0 0 0</pose>
    <mass>0.1</mass>
    <inertia>
      <ixx>0.001</ixx>
      <ixy>0</ixy>
      <ixz>0</ixz>
      <iyy>0.001</iyy>
      <iyz>0</iyz>
      <izz>0.001</izz>
    </inertia>
  </inertial>
  <visual name='visual'>
    <pose>0 0 0 0 0 0</pose>
    <geometry>
      <cylinder>
        <radius>0.025</radius>
        <length>0.025</length>
      </cylinder>
    </geometry>
     <material>
      <script>
        <uri>file://media/materials/scripts/gazebo.material</uri>
        <name>Gazebo/Grey</name>
      </script>
    </material>
  </visual>

  <sensor name="camera" type="camera">
    <pose>0 0 0 -1.57 -1.57 0</pose>
    <camera>
      <horizontal_fov>1.0472</horizontal_fov>
      <image>
        <width>640</width>
        <height>480</height>
      </image>
      <clip>
        <near>0.05</near>
        <far>1000</far>
      </clip>
    </camera>
    <always_on>1</always_on>
    <update_rate>10</update_rate>
    <visualize>true</visualize>

   <!--  <plugin name="irlock" filename="libArduCopterIRLockPlugin.so">
        <fiducial>irlock_beacon_01</fiducial>
    </plugin> -->
    <plugin name="camera_controller" filename="libgazebo_ros_camera.so">
    <alwaysOn>true</alwaysOn>
    <updateRate>0.0</updateRate>
    <cameraName>webcam</cameraName>
    <imageTopicName>image_raw</imageTopicName>
```

```
<!-- attach camera -->
<joint type="revolute" name="base_camera_joint">
  <pose>0 0 0.0 0 0 0</pose>
  <parent>iris::iris_demo::gimbal_small_2d::tilt_link</parent>
  <child>camera</child>
  <axis>
    <limit>
      <lower>0</lower>
      <upper>0</upper>
    </limit>
    <xyz>0 0 1</xyz>
    <use_parent_model_frame>true</use_parent_model_frame>
  </axis>
</joint>
```

Second sensor is lidar that is used for obstacle avoidance and like camera we need to identify lidar (link, joint)

```
<!--add lidar-->
<link name="hokuyo_link">
  <pose>0 0 0 0 0 0</pose>
  <collision name="collision">
    <pose>0 0 0.3 0 0 0</pose>
    <geometry>
      <box>
        <size>0.1 0.1 0.1</size>
      </box>
    </geometry>
  </collision>
  <visual name="visual">
    <pose>0 0 0.27 0 0 0</pose>
    <geometry>
      <mesh>
        <uri>model://hokuyo/meshes/hokuyo.dae</uri>
      </mesh>
    </geometry>
  </visual>
  <inertial>
    <mass>0.016</mass>
    <inertia>
       <ixx>0.0001</ixx>
       <ixy>0</ixy>
       <ixz>0</ixz>
       <iyy>0.0001</iyy>
       <iyz>0</iyz>
       <izz>0.0001</izz>
       <!-- low intertia necessary to avoid not disturb the drone -->
    </inertia>
  </inertial>

  <sensor type="ray" name="laser">
    <pose>0 0 0.3 0 0 1.57</pose>
```

```
      <sensor type="ray" name="laser">
        <pose>0 0 0.3 0 0 1.57</pose>
        <visualize>true</visualize>
        <update_rate>10</update_rate>
        <ray>
          <scan>
            <horizontal>
              <samples>1024</samples>
              <resolution>1</resolution>
              <min_angle>-3.141593</min_angle>
              <max_angle>3.141593</max_angle>
            </horizontal>
          </scan>
          <range>
            <min>0.1</min>
            <max>30</max>
            <resolution>0.1</resolution>
          </range>
          <!-- <noise>
            <type>Gaussian</type>
            <mean>0.0</mean>
            <stddev>0.01</stddev>
          </noise> -->
        </ray>
        <plugin name="hokuyo_node" filename="libgazebo_ros_laser.so">
          <robotNamespace></robotNamespace>
          <topicName>/spur/laser/scan</topicName>
          <frameName>/hokuyo_sensor_link</frameName>
        </plugin>
      </sensor>
    </link>

    <joint name="hokuyo_joint" type="fixed">
      <pose>0 0 0 0 0 0</pose>
      <parent>iris::iris_demo::iris::base_link</parent>
      <child>hokuyo_link</child>
    </joint>

  </model>
```

We need to create drone. launche file to launche the drone and the world in gazebo that's helping us to launch our drone with sensors in the gazebo

```
launch
  <!-- We resume the logic in empty_world.launch, changing only the name of the world to be launched -->
  <include file="$(find gazebo_ros)/launch/empty_world.launch">
    <arg name="world_name" value="$(find iq_sim)/worlds/droneOnly.world"/>
    <!-- more default parameters can be changed here -->
  </include>
</launch>
```

Finally, we need to build our work space to ensure everything is running well and reload the bashrc file in ubuntu to ensure the devel/setup.bash file is saved on it .

And new we need to open the terminal and launch the gazebo world

roslaunch iq_sim droneOnly.launch

now gazebo launch and the world appear like that

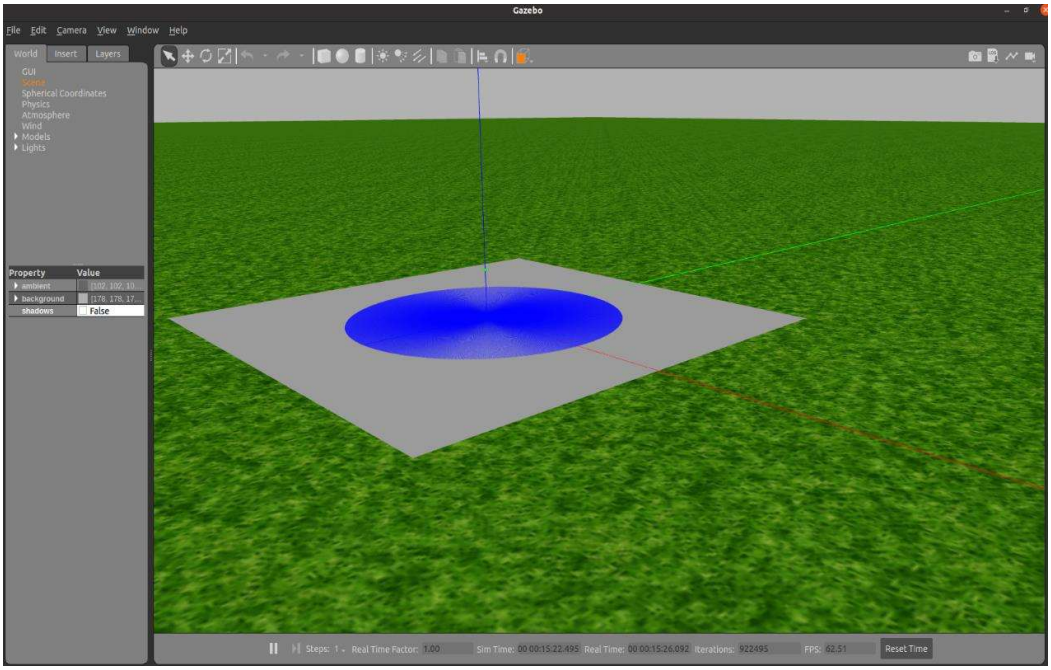

Figure 1: workspace

Now, we will create the second package for making our drone to perform some activity like moving around ground plane to detect object and the obstacle avoidance

2.IQ_gnc

First, we will create a python code for obstacle avoidance:

```
import rospy
# Import LaserScan message from package sensor_msgs.
from sensor_msgs.msg import LaserScan
# Import the API.
from iq_gnc.py_gnc_functions import *
# Import the needed math functions.
from math import cos, sin, pow, radians, sqrt
# Importing Point message from package geometry_msgs.
from geometry_msgs.msg import Point

# Create an object for the API and making it a global variable.
drone = gnc_api()


def laser_cb(msg):
    # Callback function of the subscriber.
    cr_scan = LaserScan()
    cr_scan = msg
    avoid_x = 0.0
    avoid_y = 0.0
    avoid = False

    for i in range(1, len(cr_scan.ranges)):
        d0 = 3.0
        k = 0.5
        if cr_scan.ranges[i] < d0 and cr_scan.ranges[i] > 0.35:
            avoid = True
            x = cos(cr_scan.angle_increment * i)
            y = sin(cr_scan.angle_increment * i)
            u = (-0.5 * k * pow(((1/cr_scan.ranges[i]) - (1/d0)), 2.0))

            avoid_x += (x*u)
            avoid_y += (y*u)

    # Getting the current_heading of the drone and converting it to radian
    cr_heading = radians(drone.get_current_heading())
    avoid_x = (avoid_x * cos(cr_heading)) - (avoid_y * sin(cr_heading))
    avoid_y = (avoid_x * sin(cr_heading)) + (avoid_y * cos(cr_heading))

    if avoid:
        dist = sqrt(pow(avoid_x, 2) + pow(avoid_y, 2))

        if dist > 3:
            avoid_x = (3 * (avoid_x/dist))
            avoid_y = (3 * (avoid_y/dist))

        cur_pose = Point()
        # Getting the current location from the drone.
        cur_pose = drone.get_current_location()
        # Sending the goal.
        drone.set_destination(avoid_x + cur_pose.x,
```

```
            avoid_x += (x*u)
            avoid_y += (y*u)

    # Getting the current_heading of the drone and converting it to radians.
    cr_heading = radians(drone.get_current_heading())
    avoid_x = (avoid_x * cos(cr_heading)) - (avoid_y * sin(cr_heading))
    avoid_y = (avoid_x * sin(cr_heading)) + (avoid_y * cos(cr_heading))

    if avoid:
        dist = sqrt(pow(avoid_x, 2) + pow(avoid_y, 2))

        if dist > 3:
            avoid_x = (3 * (avoid_x/dist))
            avoid_y = (3 * (avoid_y/dist))

        cur_pose = Point()
        # Getting the current location from the drone.
        cur_pose = drone.get_current_location()
        # Sending the goal.
        drone.set_destination(avoid_x + cur_pose.x,
                              avoid_y + cur_pose.y,
                              2, 0)


def main():
    # Initializing the ROS node.
    rospy.init_node("obs_avoider", anonymous=True)
    # Creating a subscriber for the topic '/spur/laser/scan'.
    rospy.Subscriber(name="/spur/laser/scan",
                     data_class=LaserScan,
                     queue_size=1,
                     callback=laser_cb)

    # Wait for FCU connection.
    drone.wait4connect()
    # Wait for the mode to be switched.
    drone.wait4start()
    # Create local reference frame.
    drone.initialize_local_frame()
    # Request takeoff with an altitude of 2m.
    drone.takeoff(2)

    # Used to keep the node running.
    rospy.spin()


if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
```

And, we can run this file by:

Rosrun iq_gnc obs_avoid.py

We can now make a script for moving drone around ground plane using square.cpp file as shown below We should move our drone using three-axis (x, y, z) that identify in real (roll, pitch, yaw)

Roll: Rotation around front-to-back axis

Pitch: Rotation around side-to-side axis

Yaw: Rotation around vertical axis.

We also should identify some function such as:

Wait4connect function: that return 0 if have a copter connect

Wait4start function: that return 0 if

The copter run the script

```
#include <gnc_functions.hpp>
//include API

int main(int argc, char** argv)
{
        //initialize ros
        ros::init(argc, argv, "gnc_node");
        ros::NodeHandle gnc_node("~");

        //initialize control publisher/subscribers
        init_publisher_subscriber(gnc_node);

        // wait for FCU connection
        wait4connect();

        //wait for used to switch to mode GUIDED
        wait4start();

        //create local reference frame
        initialize_local_frame();

        //request takeoff
        takeoff(3);

        //specify some waypoints
        std::vector<gnc_api_waypoint> waypointList;
        gnc_api_waypoint nextWayPoint;
        nextWayPoint.x = 0;
        nextWayPoint.y = 0;
        nextWayPoint.z = 3;
        nextWayPoint.psi = 0;
        waypointList.push_back(nextWayPoint);
        nextWayPoint.x = 5;
        nextWayPoint.y = 0;
        nextWayPoint.z = 3;
        nextWayPoint.psi = -90;
        waypointList.push_back(nextWayPoint);
        nextWayPoint.x = 5;
        nextWayPoint.y = 5;
        nextWayPoint.z = 3;
        nextWayPoint.psi = 0;
        waypointList.push_back(nextWayPoint);
        nextWayPoint.x = 0;
        nextWayPoint.y = 5;
        nextWayPoint.z = 3;
        nextWayPoint.psi = 90;
        waypointList.push_back(nextWayPoint);
        nextWayPoint.x = 0;
        nextWayPoint.y = 0;
        nextWayPoint.z = 3;
        nextWayPoint.psi = 180;
        waypointList.push_back(nextWayPoint);
```

```
nextWayPoint.z = 3;
nextWayPoint.psi = 0;
waypointList.push_back(nextWayPoint);
nextWayPoint.x = 5;
nextWayPoint.y = 0;
nextWayPoint.z = 3;
nextWayPoint.psi = -90;
waypointList.push_back(nextWayPoint);
nextWayPoint.x = 5;
nextWayPoint.y = 5;
nextWayPoint.z = 3;
nextWayPoint.psi = 0;
waypointList.push_back(nextWayPoint);
nextWayPoint.x = 0;
nextWayPoint.y = 5;
nextWayPoint.z = 3;
nextWayPoint.psi = 90;
waypointList.push_back(nextWayPoint);
nextWayPoint.x = 0;
nextWayPoint.y = 0;
nextWayPoint.z = 3;
nextWayPoint.psi = 180;
waypointList.push_back(nextWayPoint);
nextWayPoint.x = 0;
nextWayPoint.y = 0;
nextWayPoint.z = 3;
nextWayPoint.psi = 0;
waypointList.push_back(nextWayPoint);


//specify control loop rate. We recommend a low frequency to not over load the FCU with messages. Too many messages will cause the drone to be sluggish
ros::Rate rate(2.0);
int counter = 0;
while(ros::ok())
{
        ros::spinOnce();
        rate.sleep();
        if(check_waypoint_reached(.3) == 1)
        {
                if (counter < waypointList.size())
                {
                        set_destination(waypointList[counter].x,waypointList[counter].y,waypointList[counter].z, waypointList[counter].psi);
                        counter++;
                }else{
                        //land after all waypoints are reached
                        land();
                }
        }

}
return 0;
```

We can now run this script using the following code:

Ros run iq_gnc square.cpp

Now the last package to talk about is dark_net_ros this package allows to make object detection using yolo pretrained models but we need to install some dependencies like Cuda is a library that allows programs to take advantage of your GPU as a computing resource. YOLO will run without Cuda, but the algorithm is up to 500 x quicker with Cuda. To install Cuda, run to install cuda.

sudo apt install nvidia-cuda-toolkit

Then we clone the package in the workspace in src folder:

cd ~/drone _ws/src

git clone --recursive **https://github.com/leggedrobotics/darknet_ros.git.**

Now build the package: catkin build -DCMAKE_BUILD_TYPE=Release

we need to configure yolo in the file ros. yaml specifies ros parameters You can find this file under darknet_ros/darknet_ros/config.

You will need to change the image topic from /camera/rgb/image_raw to /webcam/image_raw.

The file darknet_ros. launch will launch the darknet/yolo ros node. You can find this file under darknet_ros/darknet_ros/launch in this file you can choose which version of yolo you would like to run by changing <arg name="network_param_file" default="$(find darknet_ros) /config/yolov2-tiny.yaml"/>.

the options are as follows:

yolov1: Not recommended. this model is old

yolov2: more accurate, and faster.

yolov3: about as fast as v2, but more accurate. Yolo v3 has a high GPU ram requirement to train and run. If your graphics card does not have enough ram, use yolo v2

tiny-yolo: Very fast yolo model. Would recommend for application where speed is most important. Works very well on Nvidia Jetson

And finally, the drone simulation will appear in gazebo like that:

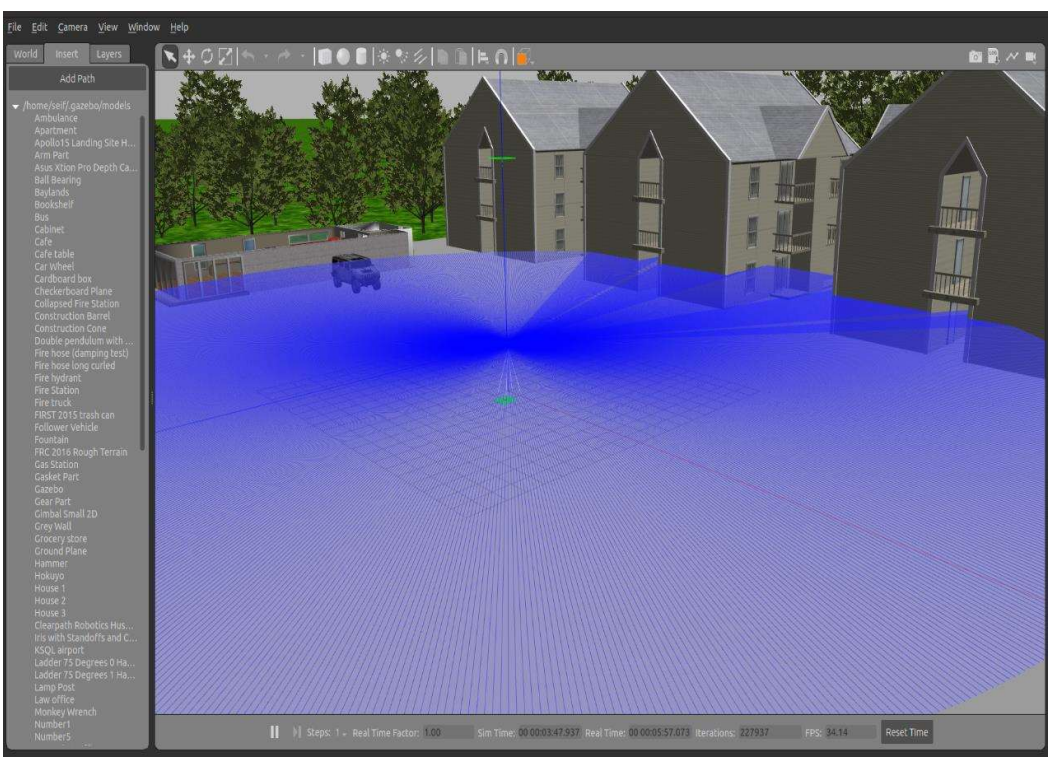

Figure 2: drone simulation